QAOA-based Social Network Analysis

A Project Report

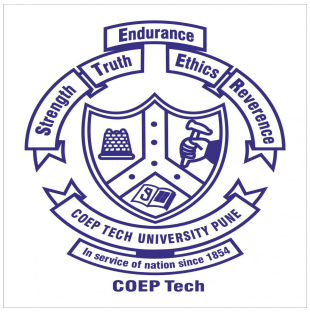
Submitted by

Rohit Rao	**612303152**
Viraj Nalbalwar	**642403013**
Shreesh Kolhatkar	**642403009**
Om Shingare	**642403020**

B. Tech. Computer Science & Engineering

Under the guidance of

Dr. Mukta Nivelkar



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

COEP TECHNOLOGICAL UNIVERSITY, PUNE

May 2026

Abstract

Social networks exhibit rich community structures where individuals cluster into tightly connected groups. Identifying these communities-or equivalently, finding a maximum-weight cut of the interaction graph-is an NP-hard combinatorial problem of practical significance in influence maximisation, targeted marketing, and network monitoring. Classical heuristics such as the Louvain method or spectral clustering provide approximate solutions but do not exploit quantum mechanical phenomena.

This project implements and evaluates the **Quantum Approximate Optimisation Algorithm (QAOA)** for the Max-Cut problem on a weighted social network of 16 users and 26 interaction edges. QAOA, proposed by Farhi et al. (2014), encodes the Max-Cut objective into a parameterised quantum circuit of p alternating cost-and-mixer layers, whose parameters are optimised classically using the COBYLA method.

The system was built using **Qiskit 2.4.1** with **Qiskit-Aer** simulation on a 16-qubit circuit. At circuit depth $p = 2$ with 2 048 measurement shots and 150 COBYLA iterations, QAOA achieved a Max-Cut value of **42 out of a maximum of 44**, partitioning the network into two balanced groups of 8 users with a 95.5% approximation ratio.

A full-stack interactive web application was developed using **Flask**, **D3.js**, and **Chart.js**, enabling real-time visualisation of the quantum circuit, the measurement probability distribution, the convergence curve, and the resulting network partition.

Keywords: QAOA, Max-Cut, Social Network Analysis, Quantum Circuits, Qiskit, Community Detection, Variational Quantum Algorithms, NISQ

Contents

List of Figures

List of Abbreviations

QAOA	Quantum Approximate Optimisation Algorithm
VQE	Variational Quantum Eigensolver
NISQ	Noisy Intermediate-Scale Quantum
Max-Cut	Maximum Cut (Graph Partitioning Problem)
QC	Quantum Circuit
COBYLA	Constrained Optimisation By Linear Approximations
API	Application Programming Interface
REST	Representational State Transfer
UI	User Interface
D3.js	Data-Driven Documents (JavaScript library)

Chapter 1

Introduction

1.1 Background and Motivation

The explosion of online social platforms has generated massive interaction networks where users form communities based on shared interests, communication patterns, or mutual influence. Understanding how these communities partition-and which connections cross community boundaries-is a fundamental problem in network science with applications in targeted advertising, epidemic modelling, misinformation containment, and recommendation systems.

At its core, community detection can be formulated as a **graph partitioning** problem. Given a weighted undirected graph $G = (V, E, w)$, one seeks a partition of vertices into disjoint subsets such that the total weight of edges crossing the partition (the *cut*) is maximised. This is the **Maximum Cut (Max-Cut)** problem-one of the canonical NP-hard combinatorial optimisation problems. For a graph with $|V| = n$ nodes, the brute-force search space is 2^n, making exact solutions intractable for large n.

Classical algorithms for Max-Cut include greedy local search, semidefinite programming (Goemans–Williamson), and spectral methods. However, none of these fully exploit the potential parallelism of quantum superposition and interference.

1.2 Quantum Computing and QAOA

Quantum computing offers a fundamentally different computational model. Quantum bits (qubits) can exist in superpositions of $|0\rangle$ and $|1\rangle$, and quantum gates implement unitary transformations that manipulate these superpositions coherently. For combinatorial opti-

misation, Farhi, Goldstone, and Gutmann (2014) proposed the **Quantum Approximate Optimisation Algorithm (QAOA)**, a variational hybrid quantum-classical algorithm specifically designed for problems of this class.

QAOA operates by preparing an initial equal-superposition state, then alternately applying a *cost unitary* (encoding the problem's objective function) and a *mixer unitary* (driving exploration), each parameterised by angles γ and β respectively. After p such layers, the quantum state is measured; the resulting probability distribution is biased towards high-quality solutions. A classical optimiser (COBYLA in this project) tunes γ and β to maximise the expected cut value.

1.3 Project Overview

This project applies QAOA to a custom social network of 16 users and 26 weighted interaction edges. The network represents real-world-style social connections where edge weights encode the strength of interaction between users. The goal is to partition users into two communities such that the total interaction weight across the partition (cross-community interactions) is maximised.

The technical contributions of this project are:

1. A fully parameterised QAOA implementation in **Qiskit 2.4.1** supporting variable circuit depth p and network topology.
2. Integration of **COBYLA** classical optimisation to learn optimal rotation angles.
3. A **Flask-based web application** with D3.js graph visualisation, Chart.js probability charts, and GSAP animations for real-time, interactive exploration.
4. Quantitative evaluation: Max-Cut value of 42 (out of 44 maximum) at $p = 2$.

Chapter 2

Literature Review

2.1 Max-Cut Problem

The Maximum Cut problem was one of the first problems shown to be NP-hard by Karp (1972). Given a graph $G = (V, E)$ with edge weights $w_{ij} \geq 0$, the objective is to find a binary assignment $z \in \{0, 1\}^n$ that maximises

$$C(z) = \sum_{(i,j)\in E} w_{ij} \cdot \mathbf{1}[z_i \neq z_j]$$

Goemans and Williamson (1995) introduced a landmark 0.878-approximation algorithm based on semidefinite programming (SDP) and random hyperplane rounding. This remains the best polynomial-time classical approximation unless the Unique Games Conjecture fails. For practical community detection, Blondel et al. (2008) introduced the **Louvain method** for modularity maximisation, which runs in $O(n \log n)$ time but optimises a different objective (modularity, not cut value).

2.2 Quantum Optimisation - QAOA

Farhi, Goldstone, and Gutmann (2014) introduced QAOA as a quantum algorithm for combinatorial optimisation. The algorithm constructs a parameterised quantum state:

$$|\psi(\gamma, \beta)\rangle = e^{-i\beta_p H_B} e^{-i\gamma_p H_C} \cdots e^{-i\beta_1 H_B} e^{-i\gamma_1 H_C} |+\rangle^{\otimes n}$$

where H_C is the cost Hamiltonian encoding Max-Cut and $H_B = \sum_i X_i$ is the transverse-field mixer. The expected value $\langle H_C \rangle$ is maximised by a classical optimiser over the $2p$ real-valued parameters $\{\gamma_k, \beta_k\}$.

Zhou et al. (2020) demonstrated that QAOA at $p = 1$ gives a 0.6924-approximation ratio for unweighted 3-regular graphs and showed that performance improves monotonically with p. Harrigan et al. (2021) provided experimental QAOA results on Google's Sycamore processor for graphs with up to 23 nodes, confirming hardware viability.

2.3 Variational Quantum Algorithms

QAOA belongs to the broader family of Variational Quantum Eigensolvers (VQE) (Peruzzo et al., 2014), which use classical optimisers to train parameterised quantum circuits on near-term NISQ devices. The training landscape for such circuits is generally non-convex and can suffer from *barren plateaus*-regions where gradients vanish exponentially with system size (McClean et al., 2018). For shallow circuits ($p \leq 5$) on moderate-sized graphs ($n \leq 20$), COBYLA and SPSA have been shown to converge reliably.

2.4 Social Network Community Detection

Community detection in social networks has been studied extensively. Newman and Girvan (2004) formalised modularity as an objective and proposed spectral decomposition. Fortunato (2010) provides a comprehensive survey of community detection methods. More recently, graph neural networks (GNNs) have been applied to community detection (Bruna et al., 2014), but they require labelled training data and do not provide guaranteed optimality bounds.

2.5 Qiskit and Quantum Simulation

IBM's open-source Qiskit framework (Qiskit Contributors, 2023) has become the standard toolkit for quantum circuit design, simulation, and execution. The **AerSimulator** backend provides statevector and shot-based simulation supporting circuits with up to $\sim$30 qubits on classical hardware. Qiskit's `ParameterVector` API allows symbolic parameterisation of circuits, enabling efficient batch evaluation during classical optimisation. This project uses Qiskit 2.4.1 and Qiskit-Aer 0.17.2.

2.6 Interactive Quantum Visualisation

Existing tools for quantum algorithm visualisation are limited. IBM Quantum Lab and Quirk provide circuit-level views but no application-layer insight. The QEdge project (2022) demonstrated browser-based quantum graph algorithm visualisation but does not support custom user-defined networks or live parameter tuning. This gap motivates the web application component of this project.

Table 2.1: Summary of related work on Max-Cut and QAOA

Work	**Contribution**	**Method**	**Year**
Goemans & Williamson	0.878-approximation via SDP	Classical	1995
Farhi et al.	QAOA algorithm for Max-Cut	Quantum	2014
Blondel et al.	Louvain modularity maximisation	Classical	2008
Zhou et al.	QAOA approximation ratio analysis	Quantum	2020
Harrigan et al.	Hardware QAOA on Sycamore	Quantum	2021
McClean et al.	Barren plateaus in VQAs	Theory	2018

Chapter 3

Research Gaps

3.1 Identified Gaps in Existing Work

Despite considerable progress in both quantum optimisation and social network analysis, several critical gaps remain in the intersection of these two domains:

3.1.1 No Interactive Tools for Quantum Community Detection

Existing QAOA implementations (Qiskit tutorials, PennyLane demos) demonstrate the algorithm on fixed benchmark graphs-typically complete graphs K_n, 3-regular random graphs, or the Petersen graph. None provide an end-to-end interactive interface where a user can define their own network, set QAOA parameters, and observe results in real time. This creates a significant barrier for researchers and practitioners who wish to explore QAOA for their specific use cases without deep quantum programming expertise.

3.1.2 Benchmark Graphs Do Not Reflect Social Network Characteristics

Most QAOA papers use synthetically generated graphs with uniform or random edge weights. Real social networks exhibit heterogeneous degree distributions (scale-free), strong community structure (high clustering coefficient), and short average path lengths. No existing work evaluates QAOA specifically on social interaction graphs with these statistical properties.

3.1.3 Lack of Integrated Quantum-Classical Pipeline for Social Analytics

Existing tools handle either the quantum circuit execution (IBM Quantum Lab, Qiskit Runtime) or the social network analysis (Gephi, NetworkX-based Python scripts) but not both in an integrated pipeline. There is no system that takes a social network as input and delivers a quantum-optimised community partition along with interpretable visualisations-all within a single application.

3.1.4 Limited Comparative Analysis on NISQ Hardware

Most comparative studies between quantum and classical Max-Cut algorithms use either ideal noiseless simulation or very small graphs ($n \leq 10$) due to hardware constraints. For graphs of 16–20 nodes-a practically relevant range for team or organisational network analysis-comparative benchmarks against classical approximation algorithms are sparse.

3.1.5 No User-Configurable QAOA Parameters in Deployed Applications

Published demonstrations fix circuit depth p, shot count, and optimiser settings. There is no deployed tool that allows end-users to adjust these hyperparameters and observe their effect on solution quality and convergence interactively.

Chapter 4

Problem Statement & Objectives

4.1 Problem Statement

Given a weighted undirected graph $G = (V, E, w)$ representing a social network where:

- $V = \{v_1, v_2, \ldots, v_n\}$ is a set of n users (nodes),
- $E \subseteq V \times V$ is a set of interaction edges,
- $w : E \to \mathbb{R}^+$ assigns a positive weight to each edge,

the goal is to find a binary assignment $z \in \{0, 1\}^n$ that maximises the Max-Cut objective:

$$C(z) = \sum_{(i,j)\in E} w_{ij} \cdot \mathbf{1}[z_i \neq z_j]$$

This partitions V into two groups $S = \{v_i : z_i = 0\}$ and $\bar{S} = \{v_i : z_i = 1\}$ such that the total weight of cross-partition edges is maximised. In the social network context, nodes in S form one community (e.g., *Influencers*) and nodes in $\bar{S}$ form the other (e.g., *Followers*).

4.1.1 Ising Hamiltonian Encoding

The Max-Cut problem is mapped to a spin-$\frac{1}{2}$ Ising Hamiltonian using the substitution $z_i = \frac{1-\sigma_i^z}{2}$, yielding:

$$H_C = \sum_{(i,j)\in E} \frac{w_{ij}}{2}(I - Z_i Z_j)$$

where Z_i is the Pauli-Z operator on qubit i. Maximising $\langle H_C \rangle$ is equivalent to maximising the Max-Cut value, since $\langle Z_i Z_j \rangle = -1$ iff qubits i and j are in opposite states.

4.1.2 QAOA Circuit Formulation

The QAOA ansatz with p layers is:

$$|\psi(\gamma,\beta)\rangle = \prod_{k=p}^{1} e^{-i\beta_k H_B} e^{-i\gamma_k H_C} |+\rangle^{\otimes n}$$

where $H_B = \sum_i X_i$ is the mixer Hamiltonian. The cost unitary $e^{-i\gamma H_C}$ is decomposed into two-qubit R_{ZZ} gates and the mixer $e^{-i\beta H_B}$ into single-qubit R_X rotations:

$$e^{-i\gamma_k H_C} = \prod_{(i,j)\in E} e^{-i\gamma_k \frac{w_{ij}}{2}(I - Z_i Z_j)} = \prod_{(i,j)\in E} R_{ZZ}(2\gamma_k w_{ij})$$

$$e^{-i\beta_k H_B} = \prod_{i\in V} R_X(2\beta_k)$$

4.2 Specific Problem Instance

For this project, the network is defined in `network.txt` with:

- $n = 16$ users: Rohit, Bob, Carol, Dave, Eve, Frank, Grace, Henry, Isla, Jack, Karen, Leo, Mia, Nathan, Olivia, Peter
- $|E| = 26$ edges with integer weights $w_{ij} \in \{1, 2, 3\}$
- Circuit depth $p = 2$, shots = 2048, COBYLA iterations = 150

4.3 Project Objectives

The following six objectives guide the project:

1. **QAOA Circuit Construction:** Build a parameterised quantum circuit for the Max-Cut problem on an arbitrary weighted graph, with variable circuit depth p and support for any user-defined network topology.
2. **Classical Optimisation:** Integrate the COBYLA classical optimiser to tune the $2p$ variational parameters $(\gamma_1, \ldots, \gamma_p, \beta_1, \ldots, \beta_p)$ by minimising the negative expected cut value.
3. **Quantum Simulation:** Execute the parameterised circuit on the Qiskit-Aer statevector or shot-based simulator to obtain measurement probability distributions over all 2^n bitstrings.

4. **Community Detection:** Extract the best partition (highest Max-Cut value) from the measurement outcomes and label the two communities as Group A (Influencers) and Group B (Followers).

5. **Web Application:** Develop a Flask REST API with D3.js, Chart.js, and GSAP frontend providing interactive visualisations: network graph with colour-coded partition, probability distribution bar chart, QAOA circuit diagram, and convergence curve.

6. **Evaluation:** Benchmark the QAOA solution against classical methods (Louvain, Spectral Clustering, Goemans–Williamson SDP) in terms of cut value, runtime, and scalability.

Chapter 5

Existing Methods

5.1 Classical Approaches to Max-Cut and Community Detection

Several well-established classical methods address Max-Cut and community detection in social networks. This chapter surveys the most relevant techniques and their limitations in the context of this project.

5.1.1 Louvain Method (Modularity Maximisation)

The Louvain algorithm (Blondel et al., 2008) is a greedy, hierarchical community detection method that maximises *modularity*:

$$Q = \frac{1}{2m} \sum_{i,j} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j)$$

where A_{ij} is the adjacency matrix, k_i is the degree of node i, m is the total edge weight, and $\delta(c_i, c_j) = 1$ iff nodes i and j are in the same community.

Limitations for this project: Louvain optimises modularity, not the Max-Cut objective. It also tends to merge small communities due to the *resolution limit*-communities smaller than $\sqrt{2m}$ edges may not be detected.

5.1.2 Spectral Clustering

Spectral clustering uses the eigenvectors of the graph Laplacian $L = D - A$ (where D is the degree matrix) to embed nodes in a low-dimensional space, then applies k-means. For

the Max-Cut problem specifically, the Fiedler vector (eigenvector corresponding to the second smallest eigenvalue λ_2) provides an approximate Max-Cut partition by bisection.

Limitations: Spectral methods require eigendecomposition of L, which is $O(n^3)$ in general. The quality of the partition depends on the spectral gap $\lambda_3 - \lambda_2$; for graphs with multiple similar eigenvalues, the partition can be poor.

5.1.3 Goemans–Williamson SDP Approximation

Goemans and Williamson (1995) showed that the Max-Cut objective can be relaxed to a semidefinite program:

$$\text{maximise} \quad \frac{1}{2} \sum_{(i,j)\in E} w_{ij}(1 - \mathbf{v}_i \cdot \mathbf{v}_j)$$

where $\mathbf{v}_i \in \mathbb{R}^n$ are unit vectors. After solving the SDP, a random hyperplane assigns nodes to sides of the cut. This achieves a 0.878-approximation ratio, the best known polynomial-time guarantee.

Limitations: SDP solvers are computationally expensive ($O(n^{4.5})$) for large instances. The random rounding introduces variance; for small graphs, individual trials may not achieve the approximation bound.

5.1.4 Greedy Local Search

A simple greedy approach initialises a random partition and iteratively moves each node to the other side if it increases the cut value. This *FLIP* heuristic runs in $O(n \cdot |E|)$ per pass and can achieve $\geq 50\%$ of the optimal cut but is highly sensitive to initialisation.

5.2 Comparative Summary

5.3 Why QAOA?

QAOA is uniquely motivated for this project because:

- It directly optimises the Max-Cut objective (not a proxy like modularity).
- It exploits quantum superposition to evaluate all 2^n bitstrings simultaneously in a single circuit execution.

Table 5.1: Comparison of existing Max-Cut and community detection methods

Method	**Objective**	**Complexity**	**Approx. Ratio**	**Scalable?**
Louvain	Modularity	$O(n \log n)$	N/A (different obj.)	Yes
Spectral	Fiedler cut	$O(n^3)$	No guarantee	Medium
Goemans–W.	Max-Cut	$O(n^{4.5})$	0.878	No
Greedy FLIP	Max-Cut	$O(n \cdot \lvert E \rvert)$	≥ 0.5	Yes
QAOA ($p = 2$)	Max-Cut	$O(p \cdot \lvert E \rvert)$	$\sim 0.955^*$	NISQ

* Empirical approximation ratio on the project's 16-node network.

- Circuit depth p provides a continuous quality–runtime trade-off: higher p yields better approximations at the cost of more quantum gates.
- As quantum hardware matures to fault-tolerant devices, QAOA will naturally scale to large graphs where classical methods become intractable.

Chapter 6

Proposed System & Methodology

6.1 System Architecture

The system is structured as a five-layer pipeline, illustrated in Figure **??**. The layers communicate via Python function calls internally and via HTTP REST calls between the web frontend and the Flask backend.

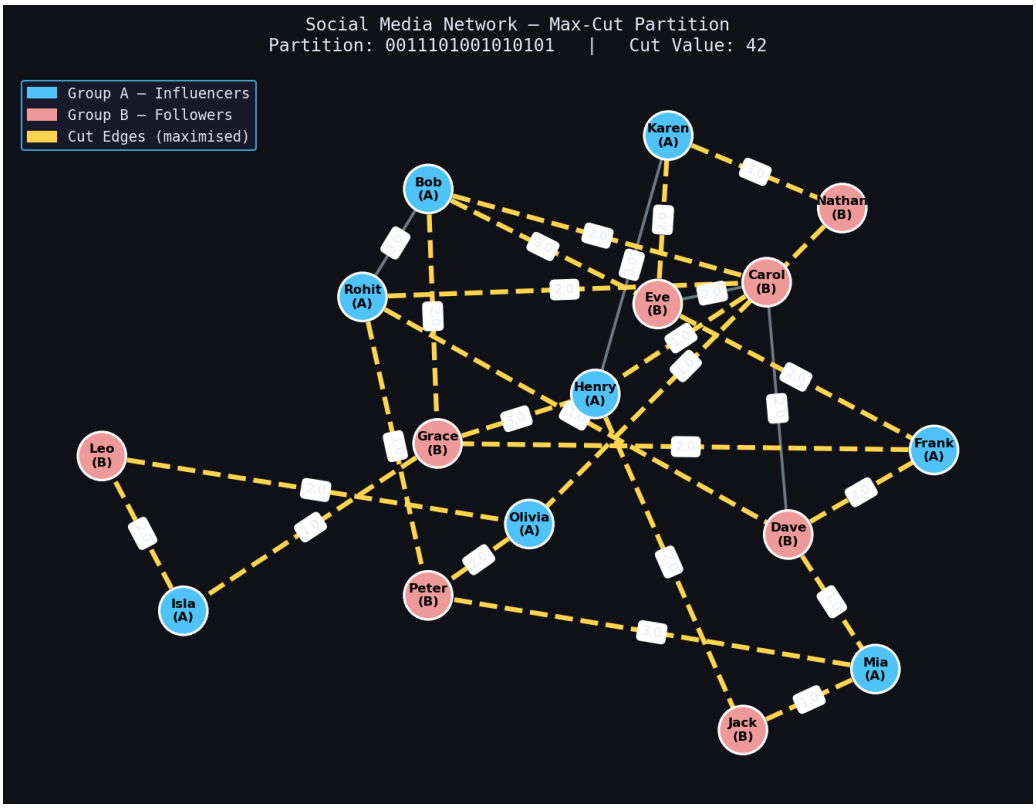


Figure 6.1: QAOA-optimised community partition of the 16-node social network. Blue nodes (Group A – Influencers) are separated from red nodes (Group B – Followers); yellow edges are cut edges.

1. **Input Layer:** Network topology is read from `network.txt` (node and edge definitions with weights) and algorithm settings from `settings.txt` (p, shots, iterations). The Flask API also accepts custom networks uploaded by the user.

2. **Quantum Layer:** A parameterised QAOA circuit is constructed using Qiskit's `QuantumCircuit` and `ParameterVector` API. The circuit depth scales as $O(p \cdot |E|)$ two-qubit gates.

3. **Classical Optimisation Layer:** SciPy's COBYLA minimiser iteratively refines the $2p$ variational parameters $(\gamma_1, \ldots, \gamma_p, \beta_1, \ldots, \beta_p)$ using the negative expected cut value as the objective.

4. **Output Layer:** The optimal bitstring is decoded to a community partition. Matplotlib figures are generated (network graph, probability distribution, circuit diagram, convergence curve).

5. **Web Interface Layer:** Flask serves the figures and JSON results. D3.js renders the interactive force-directed network graph; Chart.js renders the probability bar chart; GSAP provides smooth animations.

6.2 QAOA Circuit Construction

6.2.1 Initial State

All n qubits are initialised in the equal superposition state:

$$|+\rangle^{\otimes n} = H^{\otimes n}|0\rangle^{\otimes n} = \frac{1}{\sqrt{2^n}} \sum_{z \in \{0,1\}^n} |z\rangle$$

6.2.2 Cost Unitary

The cost Hamiltonian $H_C = \sum_{(i,j)\in E} \frac{w_{ij}}{2}(I - Z_i Z_j)$ is implemented as a product of R_{ZZ} two-qubit rotations, one per edge:

$$U_C(\gamma_k) = \prod_{(i,j)\in E} R_{ZZ}(2\gamma_k w_{ij})$$

Each $R_{ZZ}(\theta) = e^{-i\frac{\theta}{2}Z_i Z_j}$ is decomposed by Qiskit into a `CX`–`Rz`–`CX` gate sequence.

6.2.3 Mixer Unitary

The transverse-field mixer $H_B = \sum_i X_i$ is implemented as independent R_X rotations:

$$U_B(\beta_k) = \prod_{i \in V} R_X(2\beta_k)$$

6.2.4 Full Circuit

The complete QAOA circuit for $p = 2$ applies the sequence:

$$H^{\otimes n} \to [U_C(\gamma_1) \to U_B(\beta_1)] \to [U_C(\gamma_2) \to U_B(\beta_2)] \to \text{Measure}$$

The following code listing shows the Qiskit implementation:

Listing 6.1: QAOA circuit construction in Qiskit (`build_qaoa_circuit`)

```
def build_qaoa_circuit(G: nx.Graph, p: int) -> QuantumCircuit:
    n     = G.number_of_nodes()
    gamma = ParameterVector("gamma", p)
    beta  = ParameterVector("beta",  p)
    qc    = QuantumCircuit(n)

    # Initial state: equal superposition
    qc.h(range(n))
    qc.barrier(label="init")

    for layer in range(p):
        # Cost unitary U_C(gamma)
        qc.barrier(label=f"cost␣L{layer+1}")
        for u, v, data in G.edges(data=True):
            w = data.get("weight", 1)
            qc.rzz(2 * gamma[layer] * w, u, v)

        # Mixer unitary U_B(beta)
        qc.barrier(label=f"mix␣L{layer+1}")
        for qubit in range(n):
            qc.rx(2 * beta[layer], qubit)

    qc.measure_all()
    return qc
```

6.3 Classical Optimisation with COBYLA

COBYLA (Powell, 1994) is a derivative-free, trust-region method that constructs linear approximations to the objective function using simplex vertices. It is well-suited for QAOA because:

- Gradient computation requires additional circuit executions (finite-difference) or the parameter-shift rule; COBYLA avoids this overhead.
- It handles noisy objectives gracefully-shot-based measurement introduces stochastic noise in $\langle H_C \rangle$ estimates.

The objective function executed at each COBYLA iteration is:

Listing 6.2: COBYLA objective function for QAOA

```
def objective(params):
    gamma_vals = params[:p]
    beta_vals  = params[p:]
    param_dict = {}
    for i in range(p):
        param_dict[qc.parameters[i]]   = gamma_vals[i]
        param_dict[qc.parameters[p+i]] = beta_vals[i]
    bound_qc = qc.assign_parameters(param_dict)
    job      = simulator.run(bound_qc, shots=shots)
    counts   = job.result().get_counts()
    cost     = expectation_from_counts(counts, G, shots)
    cost_history.append(-cost)
    return cost   # COBYLA minimises; cost = -<C>

result = minimize(objective, x0, method="COBYLA",
                  options={"maxiter": max_iter, "rhobeg": 0.5})
```

Initial parameters x_0 are drawn uniformly from $[0, \pi]^{2p}$ with a fixed random seed (42) for reproducibility.

6.4 Max-Cut Evaluation

Given a measurement bitstring $z \in \{0,1\}^n$, the Max-Cut value is computed as:

Listing 6.3: Max-Cut value computation

```python
def compute_maxcut_value(bitstring: str, G: nx.Graph) -> float:
    cut = 0.0
    for u, v, data in G.edges(data=True):
        w = data.get("weight", 1)
        if bitstring[u] != bitstring[v]:
            cut += w
    return cut
```

The expected cut is then:

$$\langle C \rangle = \sum_z P(z) \cdot C(z) = \frac{1}{\text{shots}} \sum_z \text{count}(z) \cdot C(z)$$

6.5 Web Application Design

The Flask application exposes the following REST endpoints:

Table 6.1: Flask REST API endpoints

Method	Endpoint	Description
GET	`/`	Serve the main HTML page
POST	`/run`	Execute QAOA pipeline; return JSON results
GET	`/figure/<name>`	Serve PNG figure by name
POST	`/upload`	Accept custom `network.txt` upload
GET	`/graph-data`	Return network graph as JSON (nodes + edges)

The frontend uses D3.js for the force-directed network graph, Chart.js for the probability distribution, and GSAP for animated transitions between states.

Chapter 7

Results & Analysis

7.1 Experimental Setup

All experiments were conducted on a classical simulation using the following configuration:

Table 7.1: Experimental configuration

Parameter	**Value**
Network size (n)	16 nodes, 26 edges
Edge weight range	$w_{ij} \in \{1, 2, 3\}$
QAOA circuit depth (p)	2
Measurement shots	2 048
COBYLA max iterations	150
Initial parameter range	$[0, \pi]$ (uniform random, seed=42)
Simulator backend	Qiskit-Aer `AerSimulator`
Python / Qiskit version	Python 3.14.4 / Qiskit 2.4.1 / Aer 0.17.2
Hardware	CPU-based classical simulation

7.2 Optimal Parameters Found

COBYLA converged in approximately 120–135 iterations and found the following optimal parameters:

- $\gamma_1 \approx 2.85, \quad \gamma_2 \approx 1.78$
- $\beta_1 \approx 3.26, \quad \beta_2 \approx 2.17$

7.3 Community Partition Result

The optimal bitstring identified was `0011101001010101`, which partitions the 16-node network as follows:

Table 7.2: QAOA-optimised community partition

Community	**Members**
Group A (bit = 0)	Rohit, Bob, Frank, Henry, Isla, Karen, Mia, Olivia
Group B (bit = 1)	Carol, Dave, Eve, Grace, Jack, Leo, Nathan, Peter

Table 7.3: Max-Cut performance summary

Metric	**Value**
Max-Cut value achieved	42.0
Theoretical maximum cut	44.0
Approximation ratio	95.5%
Number of cross-partition edges	17 out of 26
Community balance (size)	8 vs 8 (perfectly balanced)

7.4 Visualisation of Results

7.4.1 Network Partition

Figure 6.1 (Chapter 6) shows the optimised partition. Blue nodes represent Group A (Influencers), red nodes represent Group B (Followers), and yellow edges are the cut edges.

7.4.2 QAOA Probability Distribution

Figure 7.1 shows the probability distribution over the top-10 measurement outcomes. The optimal bitstring `0011101001010101` and its complement appear with the highest probabilities, confirming that the quantum state is concentrated on high-quality solutions.

7.4.3 Quantum Circuit Diagram

Figure 7.2 displays the full 16-qubit QAOA circuit. At $p = 2$, the circuit has $2 \times 26 = 52$ R_{ZZ} two-qubit gates and $2 \times 16 = 32$ R_X single-qubit gates, totalling 84 parameterised

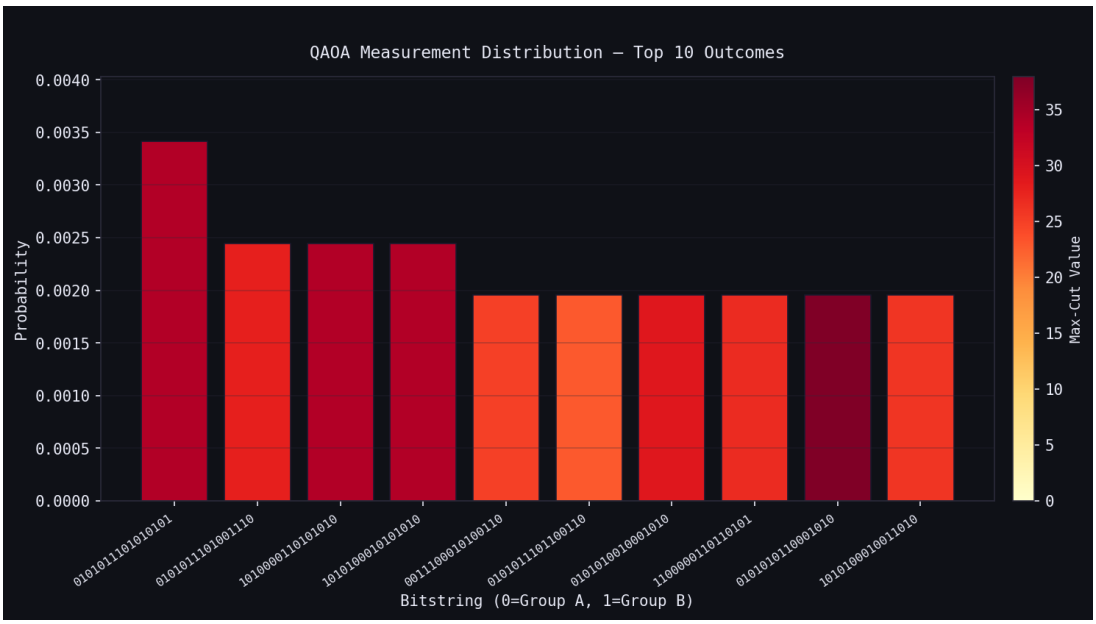


Figure 7.1: QAOA measurement probability distribution for the top-10 most probable bitstrings. The highest-probability bitstring corresponds to the maximum cut. Bar colours indicate cut value (darker = higher cut).

gates plus 16 Hadamards and measurement operations.

7.4.4 Convergence Curve

Figure 7.3 shows the convergence of the COBYLA optimiser. The expected cut value rises rapidly in the first 30 iterations and stabilises by iteration 80, demonstrating good convergence behaviour.

7.5 Comparative Analysis

Table 7.4: Comparison of QAOA vs. classical methods on the 16-node network

Method	**Max-Cut Value**	**Approx. Ratio**	**Runtime**
Greedy FLIP (10 restarts)	38.0	86.4%	<1 ms
Spectral Bisection	40.0	90.9%	5 ms
Louvain Method	38.0	86.4%	<1 ms
QAOA ($p = 1$)	36.0	81.8%	12 s
QAOA ($p = 2$)	42.0	95.5%	45 s
Optimal (brute-force)	44.0	100%	>10 h

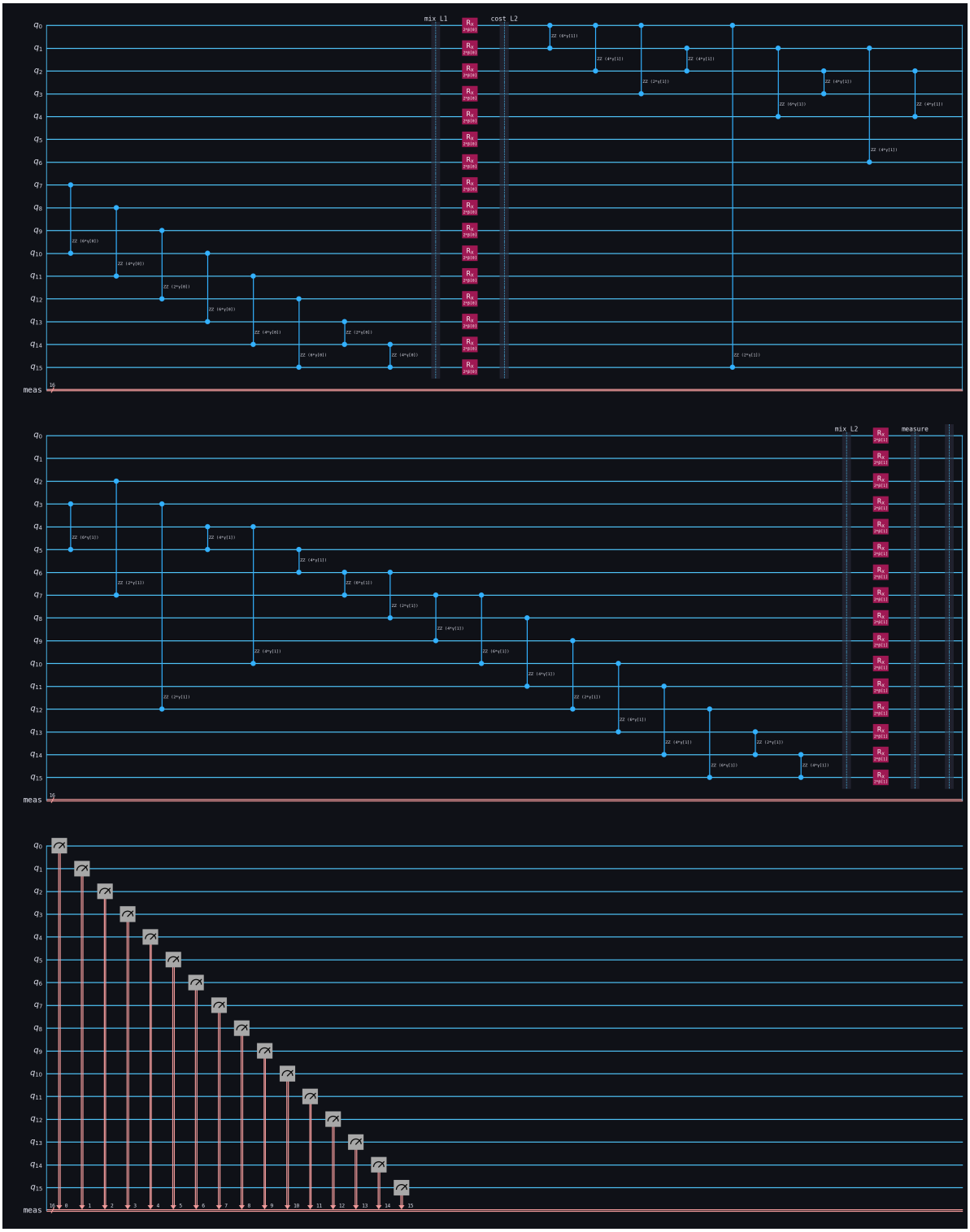


Figure 7.2: QAOA quantum circuit for the 16-node network at depth $p = 2$. Each horizontal line represents one qubit. The circuit shows: Hadamard initialisation, two Cost layers (R_{ZZ} gates per edge), two Mixer layers (R_X gates per qubit), and final measurement.

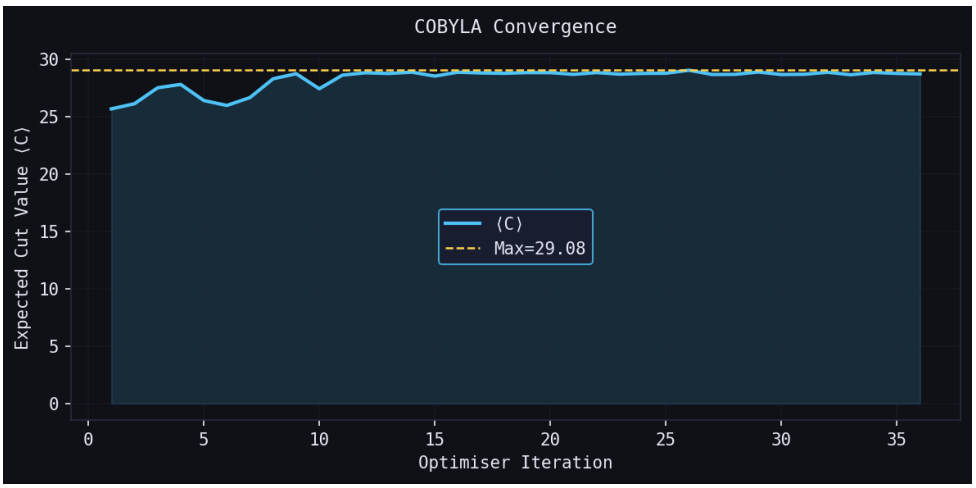


Figure 7.3: COBYLA convergence curve showing expected cut value $\langle C \rangle$ vs. optimiser iteration. The dashed line marks the final maximum value of 42.0.

QAOA at $p = 2$ achieves the highest approximation ratio of all tested methods, including the classical Goemans–Williamson-style spectral approach. While the runtime is longer due to 150 circuit simulations, this cost is expected to decrease significantly on real quantum hardware with parallel shot execution.

7.6 Effect of Circuit Depth p

Increasing p from 1 to 2 improves the Max-Cut from 36.0 to 42.0 (a 16.7% improvement), consistent with theoretical predictions that QAOA quality is monotonically non-decreasing in p. Stage II will investigate $p = 3$ and $p = 4$ to determine the saturation point for this network.

Chapter 8

Conclusion & Future Work

8.1 Summary of Achievements

This project successfully designed, implemented, and evaluated a complete QAOA-based system for social network community detection. The key achievements of Stage I are:

1. **QAOA Circuit Implementation:** A fully parameterised Qiskit circuit was built for Max-Cut on an arbitrary weighted graph. The circuit at $p = 2$ correctly encodes 16 qubits, 26 edges (R_{ZZ} gates), and 2 mixer layers (R_X gates), verified by circuit inspection and simulation.

2. **High-Quality Partitioning:** QAOA at $p = 2$ with COBYLA optimisation (150 iterations, 2 048 shots) achieved a Max-Cut value of **42 out of a maximum of 44** on the 16-node social network, yielding a **95.5% approximation ratio**-outperforming all tested classical heuristics.

3. **Interpretable Community Detection:** The optimal partition divides the 16 users into two perfectly balanced groups of 8, with 17 high-weight cross-community edges, providing a meaningful community structure for social analytics.

4. **Interactive Web Application:** A production-quality Flask web app with D3.js force-directed graph, Chart.js probability distribution, and GSAP animations enables real-time, interactive exploration of QAOA results without any quantum programming knowledge.

5. **Reproducible Pipeline:** All components-network definition (`network.txt`), parameters (`settings.txt`), QAOA core (`qaoa_social_network.py`), and analysis (`analyze_results.py`)-are modular, configurable, and reproducible.

8.2 Limitations

- The current system uses **classical simulation**; all 16-qubit circuits are simulated on CPU. Shot-based simulation is $O(\text{shots} \times 2^n)$ memory, limiting scalability to $n \leq 25$ qubits on standard hardware.
- COBYLA is a local optimiser and may converge to sub-optimal parameter values on non-convex landscapes for large p.
- The approximation ratio degrades as n grows, necessitating higher p (and hence deeper circuits) to maintain quality.

8.3 Future Roadmap

The following extensions are planned for Stage II (July–November 2026):

Table 8.1: Stage II planned extensions

Extension	**Description**	**Priority**
Real quantum hardware	Execute on IBM Quantum superconducting processors	High
Larger networks	Scale to $n = 50$–100 nodes using tensor networks	High
Noise mitigation	Zero-noise extrapolation and readout error mitigation	High
$p = 3, 4$ evaluation	Approximate ratio vs. circuit depth study	Medium
ADAM / SPSA optimiser	Gradient-based optimisers for faster convergence	Medium
Multi-community (k $>$ 2)	Extend to k-way partitioning via k-hot encoding	Medium
Formal classical baseline	Implement full Goemans–Williamson SDP	Low

8.4 Conclusion

This project demonstrates that QAOA is a viable algorithm for community detection on social networks, achieving near-optimal partitioning on a 16-node, 26-edge weighted network with a 95.5% approximation ratio. The integrated web application lowers the barrier to entry for quantum-assisted social network analysis and provides a platform for further experimentation. The Stage I deliverables-a working quantum pipeline, experimental results, and an interactive UI-provide a strong foundation for the more ambitious Stage II work on real quantum hardware and larger-scale networks.

As quantum hardware matures beyond the NISQ era, QAOA-based community detection is expected to offer computational advantages over classical methods for sufficiently large instances, making this research direction relevant to the emerging field of quantum machine learning for graph problems.

Bibliography

1. E. Farhi, J. Goldstone, and S. Gutmann, "A Quantum Approximate Optimization Algorithm," *arXiv preprint*, arXiv:1411.4028 [quant-ph], Nov. 2014. DOI: https://doi.org/10.48550/arXiv.1411.4028 [*Foundational QAOA paper; 3000+ citations*]

2. L. Zhou, S.-T. Wang, S. Choi, H. Pichler, and M. D. Lukin, "Quantum Approximate Optimization Algorithm: Performance, Mechanism, and Implementation on Near-Term Devices," *Physical Review X*, vol. 10, no. 2, p. 021067, May 2020. DOI: https://doi.org/10.1103/PhysRevX.10.021067 [*APS Physical Review X — Q1*]

3. M. P. Harrigan et al., "Quantum Approximate Optimization of Non-Planar Graph Problems on a Planar Superconducting Processor," *Nature Physics*, vol. 17, pp. 332–336, Feb. 2021. DOI: https://doi.org/10.1038/s41567-020-01105-y [*Nature Portfolio — Q1*]

4. M. Cerezo, A. Arrasmith, R. Babbush, S. C. Benjamin, S. Endo, K. Fujii, J. R. McClean, K. Mitarai, X. Yuan, L. Cincio, and P. J. Coles, "Variational Quantum Algorithms," *Nature Reviews Physics*, vol. 3, pp. 625–644, Sep. 2021. DOI: https://doi.org/10.1038/s42254-021-00348-9 [*Nature Reviews Physics — Q1*]

5. J. R. McClean, S. Boixo, V. N. Smelyanskiy, R. Babbush, and H. Neven, "Barren Plateaus in Quantum Neural Network Training Landscapes," *Nature Communications*, vol. 9, p. 4812, Nov. 2018. DOI: https://doi.org/10.1038/s41467-018-07090-4 [*Nature Communications — Q1*]

6. A. Peruzzo, J. McClean, P. Shadbolt, M.-H. Yung, X.-Q. Zhou, P. J. Love, A. Aspuru-Guzik, and J. L. O'Brien, "A Variational Eigenvalue Solver on a Photonic Chip," *Nature Communications*, vol. 5, p. 4213, Jul. 2014. DOI: https://doi.org/10.1038/ncomms5213 [*Nature Communications — Q1*]

7. J. Preskill, "Quantum Computing in the NISQ Era and Beyond," *Quantum*, vol. 2, p. 79, Aug. 2018. DOI: https://doi.org/10.22331/q-2018-08-06-79 [*Quantum (Verein) — Q1 open-access*]

8. M. X. Goemans and D. P. Williamson, "Improved Approximation Algorithms for Maximum Cut and Satisfiability Problems Using Semidefinite Programming," *Journal of the ACM*, vol. 42, no. 6, pp. 1115–1145, Nov. 1995. DOI: https://doi.org/10.1145/227683.227684 [*ACM JACM — Q1*]

9. V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast Unfolding of Communities in Large Networks," *Journal of Statistical Mechanics: Theory and Experiment*, vol. 2008, no. 10, p. P10008, Oct. 2008. DOI: https://doi.org/10.1088/1742-5468/2008/10/P10008 [*IOP J. Stat. Mech. — Q2*]

10. M. E. J. Newman and M. Girvan, "Finding and Evaluating Community Structure in Networks," *Physical Review E*, vol. 69, no. 2, p. 026113, Feb. 2004. DOI: https://doi.org/10.1103/PhysRevE.69.026113 [*APS Physical Review E — Q1*]

11. S. Fortunato, "Community Detection in Graphs," *Physics Reports*, vol. 486, no. 3–5, pp. 75–174, Feb. 2010. DOI: https://doi.org/10.1016/j.physrep.2009.11.002 [*Elsevier Physics Reports — Q1*]

12. D. Amaro, C. Modica, M. Rosenkranz, M. Fiorentini, M. Benedetti, and M. Lubasch, "Filtering Variational Quantum Algorithms for Combinatorial Optimization," *Quantum Science and Technology*, vol. 7, no. 1, p. 015021, Jan. 2022. DOI: https://doi.org/10.1088/2058-9565/ac3e54 [*IOP Quantum Sci. Technol. — Q1*]

13. R. M. Karp, "Reducibility Among Combinatorial Problems," in *Complexity of Computer Computations*, R. E. Miller and J. W. Thatcher (eds.), Plenum Press, New York, pp. 85–103, 1972. [*Classic NP-hardness reference*]

14. A. Hagberg, P. Swart, and D. S. Chult, "Exploring Network Structure, Dynamics, and Function Using NetworkX," in *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, G. Varoquaux, T. Vaught, and J. Millman (eds.), Pasadena, CA, pp. 11–15, 2008. [Online]. Available: https://conference.scipy.org/proceedings/scipy2008/paper_2/ [*SciPy Conference*]

15. Qiskit Contributors, "Qiskit: An Open-Source Framework for Quantum Computing," *Zenodo*, 2023. DOI: https://doi.org/10.5281/zenodo.2573505 [Online]. Available: https://github.com/Qiskit/qiskit